

AI Critique: Where AI Got It Wrong and What We Learned

Mistakes and Limitations

Browser Resource Management: Claude's initial automation generated mechanically correct Playwright code but completely overlooked resource management at scale. Single test cases passed, but 15 sequential tests without browser context cleanup caused timeouts. AI excels at generating isolated units but struggles with systems-level constraints (memory, async accumulation, garbage collection). The fix required human understanding of how Playwright manages browser lifecycles and the specific resource limits of the test environment.

Race Condition on Async Refetch: The backend `updateOrderStatus()` executes `await axios.put(...); fetchData()` without awaiting the refetch. AI generated code that waited only for the PUT response, then immediately read the label — a race condition. The model assumed that returning from an async function meant all downstream effects were complete, but JavaScript's async/await doesn't guarantee that un-awaited calls have finished. This required reading backend source code and understanding the specific implementation pattern, not just the API contract.

Missing Integration Testing Discipline: AI can generate individual test cases but doesn't naturally think about full-suite integration. It wrote tests that passed in isolation but failed under load. This mirrors a common AI limitation: local correctness doesn't imply global consistency. A human tester would have run the full suite early and caught accumulation effects.

Incomplete Domain Understanding: When attempting to automate the access-control bug (BUG-14), AI proposed testing via UI but missed that the Admin frontend has a client-side role gate. It didn't understand that security can be layered and that UI gates don't constitute real access control. This required domain knowledge about security theater and defense-in-depth.

What Worked Well

Code Generation and Scaffolding: AI excels at generating boilerplate (config files, data-driven JSON structures, test templates). Playwright config, HTML reporter setup, CSS selectors — all generated correctly on first try.

Refactoring Under Guidance: Once shown the patterns (context cleanup, response waits, helper extraction), AI successfully applied them throughout the codebase with minimal rework. It's good at

following established patterns but bad at discovering them.

Documentation: AI wrote clear bug reports and test descriptions without needing extensive revision. The structured markdown templates were filled accurately once the human established the structure.

Lessons Learned

1. **AI generation is not testing.** Code that compiles and runs isolated tests proves nothing about integration, performance, or resource management. Full-suite execution and load testing are essential validation steps that AI alone cannot do.
2. **Domain boundaries matter.** AI lacks understanding of system design decisions (why Checkout doesn't include shipping_address, why admin has client-side gate, why fetchData isn't awaited). Human code review must validate architecture, not just syntax.
3. **Automated correctness ≠ spec correctness.** Tests can be mechanically sound but assert against the wrong baseline (API reality instead of spec requirements). BUG-01 revealed this: the test was correct, but the implementation was wrong – and only testing against the spec, not current behavior, caught it.
4. **Resource limits are real constraints.** Sandboxed environments (GitHub CI, course VMs) have memory/CPU bounds that AI doesn't optimize for. Human experience recognizing "timeouts" as resource exhaustion, not logic errors, was critical.